ml-lab

Structured ML Hypothesis Investigation

Stop guessing whether your ML result is real. Run it through a structured adversarial review before you commit.

A Claude Code plugin — install once, use on any hypothesis

The Problem with Single-Pass Evaluation

Ask one AI to review your model result. It gives you a confident answer.

That answer is wrong more often than you think.

- **It accepts the framing you gave it** — if you present results optimistically, it evaluates optimistically
- **It has no incentive to argue for the work** — everything sounds like a caveat
- **It can't resolve genuinely two-sided questions** — when both positions are defensible, it picks one and sounds sure

> **Benchmark result:** Single-pass baseline scored 0.38 on 20 cases with known correct answers. It condemned every piece of valid work it was shown.

What ml-lab Is

A Claude Code plugin that runs a **12-step structured investigation** from hypothesis to production-ready recommendation.

Phase	Steps	What happens
Setup	Pre-flight	Sharpen hypothesis, agree on metrics, set report mode
PoC	1–2	Build minimal runnable proof-of-concept
Debate	3–5	Adversarial critique → point-by-point defense → multi-round resolution
Experiment	6–7	Run only the agreed tests, synthesize conclusions
Report	8–9	Write report, production re-evaluation
Review	10–12	Optional peer review, coherence audit, README rewrite

Install: `/plugin marketplace add chris-santiago/ml-debate-lab` then `/plugin install claude-ml-lab@ml-debate-lab`

The Two Adversarial Agents

ml-lab dispatches two subagents with opposing mandates — then forces resolution.

Agent	Persona	Job
ml-critic	Skeptical ML engineer, applied math background	Reads the PoC cold. Finds every implicit claim the code makes but hasn't tested. Cannot critique code style or out-of-scope features.
ml-defender	The original designer	Reads the critique. Responds point-by-point: concede, rebut, or mark as empirically open. Fast concession on a real problem is valued over protracted defense.

Debate runs up to 4 rounds. Any unresolved point becomes a **pre-specified empirical test** — with explicit success and failure criteria defined before any experiment runs.

Three User-Approval Gates

ml-lab never runs off on its own at the high-stakes transitions.

Gate 1 — Before any experiment runs
Presents the full experiment plan: conditions, pre-specified verdicts, conceded critique points, baseline verification requirements. You approve before a single line of experimental code runs.

Gate 2 — Before re-opening the loop
If results falsify a debate assumption, ml-lab surfaces a structured re-opening plan: what triggered it, whether to return to adversarial review (Outcome B) or reformulate the hypothesis entirely (Outcome C). You decide.

Gate 3 — Before addressing peer review findings
After the first peer review pass, ml-lab presents a remediation plan for every MAJOR and MINOR issue before touching the report.

> **Why gates?** Each one commits significant compute and shapes everything that follows. The cost of pausing is low; the cost of an unwanted macro-iteration is high.

Invoking ml-lab

Via slash command:

```
/ml-lab
```

Via natural language — describe your hypothesis and Claude Code routes automatically:

```
I think user session embedding similarity can predict churn better
than raw feature models. Can you investigate this?
```

ml-lab then asks three questions before writing any code:

1. Sharpen the hypothesis into a falsifiable claim with a named mechanism
2. Confirm the primary evaluation metric(s)
3. Choose report mode: **full report** (Steps 1–12) or **conclusions only** (Steps 1–7, 9)

Example: Fraud Detection Hypothesis

> *"An LSTM on ordered transaction sequences outperforms a bag-of-categories baseline because fraud exhibits characteristic temporal patterns."*

What happened:

- PoC returned AP = 0.96 — strong-looking
- Critic: 4 issues. Defender: conceded 3, marked 1 empirically open
- Gate 1 approved: 3-condition experiment + precondition check
- Condition C returned AP = 1.00 → suspicion trigger fired
- Soft-sort fix: AP = 0.996 — still suspicious
- **Outcome C triggered:** the equalized-distribution test is fundamentally broken for synthetic data — any ordering is trivially detectable
- Hypothesis reformulated to target the *specific fraud signature*, not just temporal ordering

Key result: The agent correctly distinguished a fixable experimental flaw from a hypothesis-level problem — without additional guidance.

What ml-lab Produces

At the end of a full investigation, you have a complete artifact record:

Artifact	Contents
`HYPOTHESIS.md`	Falsifiable claim, metric, revision history
`CRITIQUE.md` / `DEFENSE.md` / `DEBATE.md`	Full adversarial exchange
`CONCLUSIONS.md`	Per-experiment findings with pre-specified verdicts
`REPORT.md`	Self-contained technical report
`REPORT_ADDENDUM.md`	Production re-evaluation
`PEER_REVIEW_R*.md`	Structured peer review rounds (optional)
`TECHNICAL_REPORT.md`	Final synthesis in results mode (optional)

Every artifact is written before the next step begins — the record is complete even if the investigation stops early.

When to Use ml-lab

Use ml-lab when:

- You need to know *what experiment to run next*, not just whether something is broken
- You're evaluating work that might be valid and need a voice that argues *for* it
- The question is genuinely two-sided — both positions are defensible and need forcing

Use a simpler ensemble when:

- You need a quick verdict on whether something is broken
- You don't need a test specification or full report
- Speed and simplicity matter more than structured resolution

> **Benchmark:** Compute-matched ensemble scores 0.75 vs. ml-lab's 0.97. For fault detection, the ensemble gets you most of the way there. The gap is in contested-position resolution and structured argumentation.